

🔧 Endpoints Pendientes de Implementación

Guía técnica para implementar los 3 endpoints críticos que el frontend consume pero el backend no provee

de ejemplo incluido 3 bugs críticos Backend + Frontend Código

1 GET /api/v1/publicaciones/mis-publicaciones

Archivo frontend: frontend/src/pages/MisPublicaciones.jsx:22

Síntoma: La página "Mis Publicaciones" siempre usa datos mock porque la API falla.

✓ **Solución** — backend/src/routes/publicaciones.routes.js

```
// AGREGAR ESTA LÍNEA ANTES DE GET /:id
r.get('/mis-publicaciones', verificarToken, soloRoles('productor'), c.misPublicaciones);

// La ruta /:id debe ir DESPUÉS para que no atrape "mis-publicaciones"
r.get('/:id', c.obtener);
```

✓ **Solución** — backend/src/controllers/publicaciones.controller.js

```
async function misPublicaciones(req, res, next) {
  try {
    const productor = await prisma.productor.findUnique({
      where: { usuario_id: req.user.id }
    });
    if (!productor)
      return res.status(404).json({ success: false, message: 'Perfil de productor no encontrado' });

    const data = await prisma.publicacion.findMany({
      where: { productor_id: productor.id },
      include: {
        producto: { select: { nombre: true } },
        _count: { select: { negociaciones: true } }
      },
      orderBy: { created_at: 'desc' }
    });

    res.json({ success: true, data });
  } catch(e) { next(e); }
}

module.exports = { listar, obtener, crear, actualizar, cambiarEstado, eliminar, misPublicaciones };
```

2 PUT /api/v1/auth/password

Archivo frontend: frontend/src/pages/Perfil.jsx:64

Síntoma: La pestaña "Seguridad" del perfil no puede cambiar la contraseña.

✓ **Solución** — backend/src/routes/auth.routes.js

```

router.post('/register', ctrl.register);
router.post('/login', ctrl.login);
router.get('/me', verificarToken, ctrl.me);
router.put('/password', verificarToken, ctrl.cambiarPassword);

module.exports = router;

```

✓ Solución — backend/src/controllers/auth.controller.js

```

async function cambiarPassword(req, res, next) {
  try {
    const data = await authService.cambiarPassword(req.user.id, req.body);
    res.json({ success: true, data });
  } catch(e) { next(e); }
}

module.exports = { register, login, me, cambiarPassword };

```

✓ Solución — backend/src/services/auth.service.js

```

async function cambiarPassword(usuarioId, { password_actual, password_nueva }) {
  if (!password_actual || !password_nueva)
    throw Object.assign(new Error('Ambas contraseñas son requeridas'), { status: 400 });

  if (password_nueva.length < 8)
    throw Object.assign(new Error('La nueva contraseña debe tener al menos 8 caracteres'), { status: 400 });

  const usuario = await prisma.usuario.findUnique({ where: { id: usuarioId } });
  if (!usuario)
    throw Object.assign(new Error('Usuario no encontrado'), { status: 404 });

  const ok = await bcrypt.compare(password_actual, usuario.password_hash);
  if (!ok)
    throw Object.assign(new Error('La contraseña actual es incorrecta'), { status: 401 });

  const password_hash = await bcrypt.hash(password_nueva, 10);
  await prisma.usuario.update({
    where: { id: usuarioId },
    data: { password_hash }
  });

  return { message: 'Contraseña actualizada correctamente' };
}

module.exports = { register, login, me, cambiarPassword };

```

3 GET /api/v1/dashboard/stats

Archivo frontend: frontend/src/pages/Dashboard.jsx:22

Síntoma: Las estadísticas del dashboard siempre son datos mock. No hay datos reales.

✓ Solución — backend/src/controllers/dashboard.controller.js

```

const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();

async function stats(req, res, next) {
  try {
    const userId = req.user.id;

```

```

const rol    = req.user.rol;

if (rol === 'productor') {
  const p = await prisma.productor.findUnique({ where: { usuario_id: userId } });
  const productorId = p?.id;

  const [publicaciones, negociacionesActivas, entregasPendientes, ingresosMes] = await Promise.all([
    prisma.publicacion.count({ where: { productor_id: productorId } }),
    prisma.negociacion.count({ where: { productor_id: productorId, estado: { in: ['pendiente', 'en_transito'] } }, negociacion: { productor_id: productorId } }),
    prisma.entrega.count({
      where: { estado: { in: ['pendiente', 'en_transito'] }, negociacion: { productor_id: productorId } },
    }),
    prisma.pago.aggregate({
      where: { estado: 'completado', negociacion: { productor_id: productorId } },
      _sum: { monto: true }
    }),
  ]);

  res.json({
    success: true,
    data: {
      publicaciones,
      negociaciones_activas: negociacionesActivas,
      entregas_pendientes: entregasPendientes,
      ingresos_mes: ingresosMes._sum.monto ?? 0,
    }
  });
} else {
  // Para comprador o asociación
  const c = await prisma.comprador.findUnique({ where: { usuario_id: userId } });
  const compradorId = c?.id;

  const [negociacionesActivas, entregasPendientes, gastoMes] = await Promise.all([
    prisma.negociacion.count({ where: { comprador_id: compradorId, estado: { in: ['pendiente', 'en_transito'] } }, negociacion: { comprador_id: compradorId } }),
    prisma.entrega.count({
      where: { estado: { in: ['pendiente', 'en_transito'] }, negociacion: { comprador_id: compradorId } },
    }),
    prisma.pago.aggregate({
      where: { estado: 'pendiente', negociacion: { comprador_id: compradorId } },
      _sum: { monto: true }
    }),
  ]);

  res.json({
    success: true,
    data: {
      publicaciones_vistas: 0,
      negociaciones_activas: negociacionesActivas,
      entregas_pendientes: entregasPendientes,
      gasto_mes: gastoMes._sum.monto ?? 0,
    }
  });
}
} catch(e) { next(e); }
}

module.exports = { stats };

```

```
const r = require('express').Router();
const c = require('../controllers/dashboard.controller');
const { verificarToken } = require('../middleware/auth');

r.get('/stats', verificarToken, c.stats);

module.exports = r;
```

✓ Solución — backend/src/index.js (agregar línea 15)

```
app.use('/api/v1/dashboard', require('../routes/dashboard.routes'));
```

📋 Resumen de cambios necesarios

#	ENDPOINT	ARCHIVOS A MODIFICAR	ARCHIVOS A CREAR
1	GET /api/v1/publicaciones/mis-publicaciones	routes/publicaciones.routes.js controllers/publicaciones.controller.js	—
2	PUT /api/v1/auth/password	routes/auth.routes.js controllers/auth.controller.js services/auth.service.js	—
3	GET /api/v1/dashboard/stats	index.js	routes/dashboard.routes.js controllers/dashboard.controller.js